

CuraHealth – Selenium Java Automation Testing Framework Plan

This document contains the complete implementation plan for the Selenium-Java Automation Testing Framework for the Cura Healthcare Appointment Portal. The framework follows the Page Object Model (POM), TestNG architecture, reusable utilities, reporting, screenshots, configuration management, and data-driven testing approach.

1. Project Objective

- Automate end-to-end healthcare appointment workflows.
- Build a reusable Selenium Java framework using POM.
- Validate authentication, appointment booking, history, and form validations.
- Generate Extent Reports with screenshots on failures.
- Run complete suite using Maven and TestNG.

2. Tools and Technologies

- Java
- Selenium WebDriver
- TestNG
- Maven
- WebDriverManager
- ExtentReports
- Git & GitHub
- Apache POI (optional for Excel DataProvider)
- JSON / Jackson Library (optional for JSON DataProvider)

3. Framework Architecture

- Page Object Model (POM)
- Base Classes
- Utilities
- Listeners
- Reports
- Configuration Files
- Reusable Methods
- Data-Driven Testing

4. Folder Structure

Recommended Folder Structure

```
src/test/java
```

```
├── base
```

```
|   └─ BaseTest.java
└─ pages
|   └─ LoginPage.java
|   └─ AppointmentPage.java
|   └─ ConfirmationPage.java
|   └─ HistoryPage.java
|   └─ ProfilePage.java
└─ tests
|   └─ LoginTests.java
|   └─ AppointmentTests.java
|   └─ HistoryTests.java
|   └─ MultipleAppointmentTests.java
|   └─ ValidationTests.java
└─ utilities
|   └─ ConfigReader.java
|   └─ ScreenshotUtil.java
|   └─ WaitUtil.java
|   └─ ExtentManager.java
└─ listeners
|   └─ TestListener.java
└─ data
|   └─ LoginData.json / Excel resources
└─ config.properties
└─ testng.xml
```

6. Functional Requirements

- User Login & Logout
- Appointment Booking
- Appointment Confirmation

- Appointment History Verification
- Multiple Appointment Booking
- Validation Testing

7. Non-Functional Requirements

- Reusability
- Scalability
- Maintainability
- Reporting Support
- Screenshot Capture
- Configurable Execution

8. Detailed Test Cases

TC_ID	Module	Test Scenario	Expected Result
TC_LOGIN_01	Authentication	Login with valid credentials	User should login successfully
TC_LOGIN_02	Authentication	Login with invalid credentials	Error message should display
TC_LOGIN_03	Authentication	Logout from application	User redirected to homepage
TC_LOGIN_04	Authentication	Access protected page without login	Redirected to login page
TC_APPT_01	Appointment	Book appointment with valid data	Appointment confirmation displayed
TC_APPT_02	Appointment	Book appointment with admission checkbox	Checkbox status visible in confirmation
TC_APPT_03	Appointment	Enter past date	Past date should not be accepted
TC_HIST_01	History	Verify appointment history loads	History page displayed
TC_HIST_02	History	Verify latest appointment appears	Correct appointment visible
TC_MULTI_01	Multiple Appointment	Book two appointments	Both appointments displayed
TC_VALID_01	Validation	Submit appointment form without date	Validation/error should appear
TC_VALID_02	Validation	Submit login without credentials	Validation message displayed

9. Step-by-Step Execution Plan

1. Install Java JDK and Maven
2. Create Maven Project
3. Add dependencies in pom.xml

4. Configure WebDriverManager
5. Create BaseTest and BasePage classes
6. Create Page Object classes
7. Develop reusable utility methods
8. Implement TestNG test classes
9. Add DataProvider for login tests
10. Implement screenshot listener
11. Configure ExtentReports
12. Create testng.xml
13. Execute tests using mvn test
14. Analyze reports and screenshots

10. Maven Dependencies

Required Maven Dependencies

- selenium-java
- testng
- webdrivermanager
- extentreports
- apache-poi (optional)
- jackson-databind (optional)

11. Page Object Model Design

- LoginPage.java – Login actions and validations
- HomePage.java – Navigation and logout
- AppointmentPage.java – Appointment form handling
- ConfirmationPage.java – Confirmation validation
- HistoryPage.java – Appointment history verification

12. Test Strategy

- Authentication Tests: Valid Login, Invalid Login, Logout
- Appointment Tests: Book Appointment, Admission Checkbox Validation, Past Date Validation
- History Tests: Verify Appointment History, Verify Column Headers
- Validation Tests: Empty Login Validation, Empty Date Validation

13. Data Driven Testing

- Use TestNG DataProvider to execute tests with multiple datasets.
- Valid credentials
- Invalid credentials
- Blank values

14. Wait Strategy

- Avoid Thread.sleep()
- Use WebDriverWait
- Use ExpectedConditions
- elementToBeClickable()
- visibilityOfElementLocated()

15. Screenshot & Reporting

- Capture screenshots on failure
- Store inside /screenshots folder
- HTML Report Generation
- Pass/Fail Status
- Execution Time
- Embedded Failure Screenshots

16. Listener Design

- Implement TestNG ITestListener
- Listen for test execution
- Capture screenshots on failures
- Update Extent Reports

17. TestNG Suite Design

- Suite execution
- Test classes
- Listener configuration
- Parallel execution support

18. Best Practices

- Do not use Thread.sleep() anywhere in the framework
- Use WebDriverWait and ExpectedConditions
- Keep locators only inside Page classes
- Use reusable methods for common actions
- Store screenshots in /screenshots folder
- Use meaningful method names
- Run complete suite through testng.xml
- Maintain clean and readable code

19. Conclusion

This framework plan provides a complete structure for building a scalable Selenium-Java Automation Testing Framework for the Cura Healthcare demo application. By following this approach, the team can achieve clean architecture, maintainable code, reusable components, proper reporting, and full end-to-end automation coverage.